Lab Report: Kalman Filtering and Application to Mobile Target Tracking

ANANDARAJAH Rahul

March 8, 2026

1. Problem Context

The Kalman filter is a popular filter that addresses the following problem. Consider an aircraft whose trajectory we wish to observe. The aircraft has a real trajectory and an observed trajectory. The observed trajectory may differ from the real trajectory because the observation may be degraded by camera noise, for example. The problem is therefore to determine the real trajectory from the observations.

2. Some Demonstrations

1. Classical Kalman Filter Algorithm

Using the problem parameterisation described in the lab sheet and its notations, in the first part of the lab, we propose to write an iterative Python algorithm that provides an expression for the Kalman filter at time $t+1$ given its expression at time t. Among all the data provided to write this algorithm, we are missing the formula that allows us to go from $\mathbf{m_{t|t}}$, $\mathbf{P_{t|t}}$ and the other parameters of the problem to $\mathbf{m_{t+1|t+1}}$, $\mathbf{P_{t+1|t+1}}$, which we propose to demonstrate. According to the course, denoting by p the density of the law followed by the random variable X_t:

$$X_t|Y_{0:t} \sim \mathcal{N}(\mathbf{m_{t|t}}, \mathbf{P_{t|t}}) \text{ i.e., } p(X_t|Y_{0:t}) = \mathcal{N}(X_t, \mathbf{m}_{t|t}, \mathbf{P}_{t|t})$$

Let us write the expression we wish to obtain:

$$\begin{aligned} p(X_{t+1}|Y_{0:t+1}) &= p(X_{t+1}|Y_{0:t}, Y_{t+1}) \\ &= \frac{p(Y_{t+1}|\cancel{Y_{0:t}}X_{t+1})p(X_{t+1}|Y_{0:t})}{\int p(Y_{t+1}|\cancel{Y_{0:t}}X_{t+1})p(X_{t+1}|Y_{0:t})dx_{t+1}} \text{ by Bayes' formula and the model} \end{aligned}$$

According to equation (2) of the lab sheet, $Y_{t+1} = \mathbf{H}X_{t+1} + V_{t+1}$. Conditioned on X_{t+1}, $\mathbf{H}X_{t+1}$ is deterministic, and $V_{t+1} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$, so $p(Y_{t+1}|X_{t+1}) = \mathcal{N}(Y_{t+1}, \mathbf{H}X_{t+1}, \mathbf{R})$ (A). It then follows:

$$\begin{aligned} p(X_{t+1}|Y_{0:t+1}) &= \frac{\mathcal{N}(y_{t+1}, \mathbf{H}X_{t+1}, \mathbf{R})p(X_{t+1}|Y_{0:t})}{\int \mathcal{N}(y_{t+1}, \mathbf{H}X_{t+1}, \mathbf{R})p(X_{t+1}|Y_{0:t})\, dx_{t+1}} \\ \text{Now, } p(X_{t+1}|Y_{0:t}) &= \int p(X_{t+1}, X_t|Y_{0:t})\, dx_t \text{ by the total probability formula} \\ &= \int p(X_{t+1}|X_t, \cancel{Y_{0:t}})p(X_t|Y_{0:t})\, dx_t \text{ by the model} \\ &= \int \mathcal{N}(X_{t+1}, \mathbf{F}X_t, \mathbf{Q})\mathcal{N}(X_t, \mathbf{m_{t|t}}, \mathbf{P_{t|t}})\, dx_t \text{ analogous to (A) applied to equation (1)} \end{aligned}$$

According to the first identity seen in the lab, a sketch of the proof can be found later in this report:

$$\int \mathcal{N}(\mathbf{y}, \mathbf{A}\mathbf{x} + \mathbf{b}, \mathbf{L})\mathcal{N}(\mathbf{x}, \mu, \mathbf{G})\, dx = \mathcal{N}(\mathbf{y}, \mathbf{A}\mu + \mathbf{b}, \mathbf{L} + \mathbf{A}\mathbf{G}\mathbf{A}^{\mathbf{T}}).$$

$$p(X_{t+1}|Y_{0:t}) = \mathcal{N}(X_{t+1}, \mathbf{m_{t+1|t}}, \mathbf{P_{t+1|t}}) \quad \text{where} \quad \begin{cases} \mathbf{m}_{t|t-1} & = \mathbf{F}\mathbf{m}_{t-1|t-1} \\ \mathbf{P}_{t|t-1} & = \mathbf{Q} + \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^T \end{cases}$$

According to the second identity seen in the lab:

$$\mathcal{N}(\mathbf{y}, \mathbf{H}\mathbf{x}, \mathbf{R})\mathcal{N}(\mathbf{x}, \mathbf{m}, \mathbf{P}) = p(\mathbf{y})\mathcal{N}(\mathbf{x}, \mathbf{m} + \mathbf{K}(\mathbf{y} - \mathbf{H}\mathbf{m}), (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P})$$

where $\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1}$. Therefore:

$$\begin{aligned} p(X_{t+1}|Y_{0:t+1}) &= \frac{\mathcal{N}(Y_{t+1}, \mathbf{H}X_{t+1}, \mathbf{R})\mathcal{N}(X_{t+1}, \mathbf{m_{t+1|t}}, \mathbf{P_{t+1|t}})}{\int \mathcal{N}(Y_{t+1}, \mathbf{H}X_{t+1}, \mathbf{R})\mathcal{N}(X_{t+1}, \mathbf{m_{t+1|t}}, \mathbf{P_{t+1|t}})\, dx_{t+1}} \\ &= \frac{\mathcal{N}(X_{t+1}, \mathbf{m}_{t+1|t+1}, \mathbf{P}_{t+1|t+1})}{\cancel{\int \mathcal{N}(X_{t+1}, \mathbf{m}_{t+1|t+1}, \mathbf{P}_{t+1|t+1})\, dx_{t+1}} = 1} \quad \text{where} \begin{cases} \mathbf{m_{t+1|t+1}} & = \mathbf{m_{t+1|t}} + \mathbf{K}(Y_{t+1} - \mathbf{H}\mathbf{m_{t+1|t}}) \\ \mathbf{K} & = \mathbf{P_{t+1|t}}\mathbf{H}^T(\mathbf{H}\mathbf{P_{t+1|t}}\mathbf{H}^T + \mathbf{R})^{-1} \\ \mathbf{P_{t+1|t+1}} & = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P_{t+1|t}} \end{cases} \end{aligned}$$

We implemented the filter in Python; the code is available in the appendix. The results of the experiments on the filter are available in the rest of the report.

2. Extended Kalman Filter Algorithm

In the second part of the lab, we look at extended Kalman filtering. The relation $Y_t = \mathrm{h}(X_t)$ is no longer linear and (2) is no longer valid: we can therefore no longer apply the Kalman filter as we know it. We then decide to work around the problem by making h linear using a first-order Taylor approximation. Note that we expand h to first order, because at higher orders the approximation would no longer be linear. We choose to expand h around the point $\mathbf{m_{t+1|t}}$ because since only the update step is affected and not the prediction, the best estimate of the target's position at time t+1 that we have is $\mathbf{m_{t+1|t}}$. Let us revisit the various calculation steps described above in more detail:

$$h : \begin{cases} \mathbb{R}^4 & \longrightarrow \mathbb{R}^2 \\ \begin{pmatrix} p_x \\ \dot{p}_x \\ p_y \\ \dot{p}_y \end{pmatrix} & \mapsto \begin{pmatrix} \sqrt{p_x^2 + p_y^2} \\ \arctan\left(\frac{p_y}{p_x}\right) \end{pmatrix} \end{cases} \quad \text{so} \quad J_h : \begin{cases} \mathbb{R}^4 & \longrightarrow \mathbb{R}^{2\times 4} \\ \begin{pmatrix} p_x \\ \dot{p}_x \\ p_y \\ \dot{p}_y \end{pmatrix} & \mapsto \begin{bmatrix} \frac{p_x}{\sqrt{p_x^2+p_y^2}} & 0 & \frac{p_y}{\sqrt{p_x^2+p_y^2}} & 0 \\ -\frac{p_y}{p_x^2+p_y^2} & 0 & \frac{p_x}{p_x^2+p_y^2} & 0 \end{bmatrix} \end{cases}$$

Taylor expansion of h around $\mathbf{m_{t+1|t}}$:

$$\begin{aligned} Y_t &= h(\mathbf{m_{t+1|t}}) + J_h(\mathbf{m_{t+1|t}})(X_t - \mathbf{m_{t+1|t}}) + V_t \\ &= J_h(\mathbf{m_{t+1|t}})X_t + V_t + h(\mathbf{m_{t+1|t}}) - J_h(\mathbf{m_{t+1|t}})\mathbf{m_{t+1|t}} \\ Z_t &\triangleq Y_t - h(\mathbf{m_{t+1|t}}) + J_h(\mathbf{m_{t+1|t}})\mathbf{m_{t+1|t}} \\ &= J_h(\mathbf{m_{t+1|t}})X_t + V_t \end{aligned}$$

The system (1) and (2) thus becomes:

$$\begin{cases} X_t = \mathbf{F}X_{t-1} + U_t & \text{with } U_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \\ Z_t = J_h(\mathbf{m_{t+1|t}})X_t + V_t & \text{with } V_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \end{cases}$$

Since the prediction step is preserved, we only adapt the update step in the new Kalman filter:

$$\text{Prediction:} \quad \begin{cases} \mathbf{m_{t+1|t}} &= \mathbf{F}\mathbf{m_{t|t}} \\ \mathbf{P_{t+1|t}} &= \mathbf{Q} + \mathbf{F}\mathbf{P_{t|t}}\mathbf{F}^T \end{cases}$$

$$\text{Update:} \quad \begin{cases} \mathbf{K} &= \mathbf{P_{t+1|t}} J_h(\mathbf{m_{t+1|t}})^T \left(J_h(\mathbf{m_{t+1|t}})\mathbf{P_{t+1|t}} J_h(\mathbf{m_{t+1|t}})^T + \mathbf{R} \right)^{-1} \\ \mathbf{m_{t+1|t+1}} &= \mathbf{m_{t+1|t}} + \mathbf{K}(Z_{t+1} - J_h(\mathbf{m_{t+1|t}})\mathbf{m_{t+1|t}}) \\ \mathbf{P_{t+1|t+1}} &= (\mathbf{I} - \mathbf{K} J_h(\mathbf{m_{t+1|t}}))\mathbf{P_{t+1|t}} \end{cases}$$

We implemented the new filter in Python; the code is available in the appendix. The results of the experiments on the filter are available in the following section.

3. Bonus: Application

Consider the following problem. From time to time, the target observation information is not available, either because the target is not observable or because the data has been corrupted. Denoting this instant $t-1$, we need to find the best estimate of X_t given $Y_{0:t-1}$ since Y_t is not available.

Revisiting the course: let $H = \mathcal{L}^2(\Omega, \mathcal{A}, P)$ be the Hilbert space of centred, square-integrable random variables defined on $(\Omega, \mathcal{A}, P)$ equipped with the inner product $(X, Y) = \mathbb{E}(XY)$. Finding the best estimate of X_t given $Y_{0:t-1}$ amounts to minimising the distance between X_t and the space spanned by $Y_{0:t-1}$. This distance is $\mathbb{E}(X_t|Y_{0:t-1})$ and is the orthogonal projection of X_t onto H. To compute this quantity, we will determine the expectation of the conditional distribution $X_t|Y_{0:t-1}$. Let us compute the density of this distribution:

$$\begin{aligned}
p(X_t|Y_{0:t-1}) &= \int p(X_t, X_{t-1}|Y_{0:t-1})\, dx_{t-1} \text{ by the total probability formula} \\
&= \int p(X_t|X_{t-1}, \cancel{Y_{0:t-1}})p(X_{t-1}|Y_{0:t-1})\, dx_{t-1} \text{ by the model} \\
&= \int \mathcal{N}(X_t, \mathbf{F}X_{t-1}, \mathbf{Q})\mathcal{N}(X_{t-1}, \mathbf{m}_{t-1|t-1}, \mathbf{P}_{t-1|t-1})\, dx_{t-1} \text{ analogous to (A) applied to equation (1)} \\
&= \mathcal{N}(X_t, \mathbf{m}_{t|t-1}, \mathbf{P}_{t|t-1}) \text{ by the first identity seen in the lab where } \begin{cases} \mathbf{m}_{t|t-1} &= \mathbf{F}\mathbf{m}_{t-1|t-1} \\ \mathbf{P}_{t|t-1} &= \mathbf{Q} + \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^T \end{cases}
\end{aligned}$$

Therefore, $\mathbb{E}(X_t|Y_{0:t-1}) = \mathbf{m}_{t|t-1}$. The best estimate of X_t given $Y_{0:t-1}$ corresponds to the prediction step alone in the search for the best estimate of X_t given $Y_{0:t}$. When we implement the solution in Python, we will take care to return only the prediction step of the Kalman filter without the update. Since the data from the lab sheet is unusable, we will only implement the solution without being able to test it.

4. Bonus: Proof Outline for the Convolution of Multivariate Gaussians (Identity 1)

Consider two multivariate Gaussian distributions: $\mathcal{N}(\mathbf{y}; \mathbf{A}\mathbf{x} + \mathbf{b}, \mathbf{L})$ and $\mathcal{N}(\mathbf{x}; \mu, \mathbf{G})$. We seek to demonstrate that:

$$\int \mathcal{N}(\mathbf{y}, \mathbf{A}\mathbf{x} + \mathbf{b}, \mathbf{L})\mathcal{N}(\mathbf{x}, \mu, \mathbf{G})\, d\mathbf{x} = \mathcal{N}(\mathbf{y}, \mathbf{A}\mu + \mathbf{b}, \mathbf{L} + \mathbf{A}\mathbf{G}\mathbf{A}^{\mathbf{T}}).$$

① Formulation of the Gaussian distributions:
The form of the multivariate Gaussian distribution is given by:

$$\mathcal{N}(\mathbf{x}; \mu, \mathbf{\Sigma}) = \frac{1}{\sqrt{(2\pi)^k |\mathbf{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \mathbf{\Sigma}^{-1} (\mathbf{x} - \mu) \right),$$

where $|\mathbf{\Sigma}|$ is the determinant of $\mathbf{\Sigma}$ and k is the dimension of $\mathbf{x}$.
Multiplying the two Gaussian densities, we obtain:

$$\mathcal{N}(\mathbf{y}, \mathbf{A}\mathbf{x} + \mathbf{b}, \mathbf{L})\mathcal{N}(\mathbf{x}, \mu, \mathbf{G}) = \frac{1}{(2\pi)^k \sqrt{|\mathbf{L}||\mathbf{G}|}} \exp\left(-\frac{1}{2}R\right),$$

where

$$R = (\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b})^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b}) + (\mathbf{x} - \mu)^T \mathbf{G}^{-1} (\mathbf{x} - \mu).$$

We must now integrate the expression above with respect to x to obtain the desired expression. To do this, we propose to complete the square for $\mathbf{x}$ by putting the argument of the exponential in the form:

$$-\frac{1}{2}(\mathbf{x} - \mathbf{\Sigma}_x)^T \mathbf{\Lambda} (\mathbf{x} - \mathbf{\Sigma}_x) + \text{terms in } \mathbf{y},$$

where $\mathbf{\Sigma}_x$ and $\mathbf{\Lambda}$ are derived from the original parameters.

② Completing the square:

$$\begin{aligned}
R &= (\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b})^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{A}\mathbf{x} - \mathbf{b}) + (\mathbf{x} - \mu)^T \mathbf{G}^{-1} (\mathbf{x} - \mu) \\
&= \mathbf{x}^T \mathbf{A}^T \mathbf{L}^{-1} \mathbf{A}\mathbf{x} - 2\mathbf{x}^T \mathbf{A}^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) \\
&\quad + (\mathbf{y} - \mathbf{b})^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) + \mathbf{x}^T \mathbf{G}^{-1} \mathbf{x} - 2\mathbf{x}^T \mathbf{G}^{-1} \mu + \mu^T \mathbf{G}^{-1} \mu \\
&= \mathbf{x}^T (\mathbf{A}^T \mathbf{L}^{-1} \mathbf{A} + \mathbf{G}^{-1}) \mathbf{x} - \mathbf{x}^T 2(\mathbf{A}^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) + \mathbf{G}^{-1} \mu) + (\mathbf{y} - \mathbf{b})^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) \\
&\quad + \mu^T \mathbf{G}^{-1} \mu
\end{aligned}$$

③ Combining the terms in x:
Quadratic term in x:

$$\mathbf{S} = \mathbf{A}^T \mathbf{L}^{-1} \mathbf{A} + \mathbf{G}^{-1}$$

With:

$$\mathbf{S}^{-1} = \left(\frac{1}{\sigma^2} + \frac{1}{\tau^2}\right) \mathbf{I}, \quad \text{with } \sigma^2, \tau^2 \in \mathbb{R}$$

Linear term in x:

$$-\mathbf{x}^T 2(\mathbf{A}^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) + \mathbf{G}^{-1} \mu)$$

We can rewrite it so that it equals:

$$-2\left(\frac{1}{\sigma^2}(\bar{\mathbf{x}} - \bar{\boldsymbol{\mu}})^T - \frac{1}{\tau^2}(\mathbf{A}^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}))^T\right) \mathbf{x}, \quad \text{with } \bar{\mathbf{x}}, \bar{\boldsymbol{\mu}} \in \mathbb{R}^k$$

This term can be rearranged in preparation for completing the square by isolating the coefficient of x.
Constant term:

$$(\mathbf{y} - \mathbf{b})^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) + \mu^T \mathbf{G}^{-1} \mu$$

④ Definition of new parameters:
Let us introduce the following change of variables:

$$\begin{cases} \mathbf{S} = \mathbf{A}^T \mathbf{L}^{-1} \mathbf{A} + \mathbf{G}^{-1} \\ \mathbf{m} = \mathbf{S}^{-1} (\mathbf{A}^T \mathbf{L}^{-1} (\mathbf{y} - \mathbf{b}) + \mathbf{G}^{-1} \boldsymbol{\mu}) \end{cases}$$

The Jacobian of the change of variables has a determinant equal to $|\mathbf{L} + \mathbf{A}\mathbf{G}\mathbf{A}^{\mathbf{T}}|$.

Conclusion:

Integrating this quadratic form in $\mathbf{x}$ yields a new Gaussian in $\mathbf{y}$ with adjusted mean vector and covariance matrix. Performing the integration and grouping the constant terms, we obtain the final multivariate Gaussian distribution for $\mathbf{y}$:

$$\mathcal{N}(\mathbf{y}, \mathbf{A}\mu + \mathbf{b}, \mathbf{L} + \mathbf{AGA^T}).$$

This result shows how the parameters of the resulting distribution are formed from the parameters of the two original distributions, thus completing the demonstration of the convolution of multivariate Gaussians.

3. Analysis of Results

We will focus on one particular iteration of five scenarios. Ideally, averages should be computed over each scenario for a proper analysis.

1. Ideal Scenario

Before varying each parameter individually to evaluate their effects, let us place ourselves in an ideal case with little trajectory noise and little observation noise along both axes, i.e., $(\sigma_Q, \sigma_{px}, \sigma_{py}) = (1, 3, 3)$.

In the case of the classical Kalman filter (KF), we see that the 3 trajectories almost coincide. The mean squared error between the real trajectory and the estimated trajectory is $\epsilon_{KF1} = 3.36$.

In the case of the extended Kalman filter (EKF), the curves coincide less: the real and observed trajectories coincide, since the observation is the trajectory with low observation noise, but the estimated trajectory deviates slightly from the other two. We can hypothesise that the remainder of the Taylor expansion enters the observation noise of Z_t, even though it was chosen to be low. It is therefore as if it had been chosen higher than expected in the prediction and update steps of the EKF. In theory, the EKF is locally good but not globally so because h is only locally linear. The squared error for the EKF ϵ_{EKF1}= 491.33 $>>>$ ϵ_{KF1}, which shows that the KF is much better than the EKF.

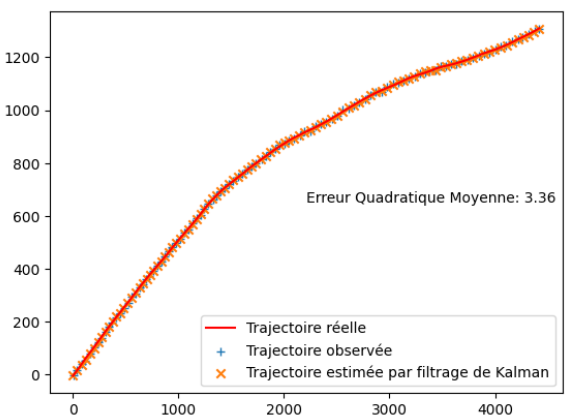

Figure 1: KF trajectories ideal scenario

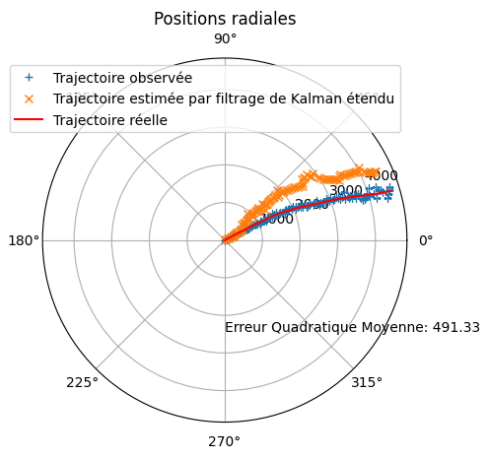

Figure 2: EKF trajectories ideal scenario

2. Noisy Trajectory Scenario

Let us consider the case $(\sigma_Q, \sigma_{px}, \sigma_{py}) = (100,\ 3,\ 3)$ to evaluate the impact of target trajectory noise. In the two previous cases, we had a linear trajectory; here it is somewhat more complicated due to the trajectory noise forcing the target to abruptly change direction at each time step.

In the case of the KF, the squared error $\epsilon_{KF2} = 69.30 > \epsilon_{KF1}$ has increased significantly compared to the ideal case but remains low. Indeed, the three curves coincide visually, the observation noises have not changed and are fairly good: it appears that the filter partially manages to compensate for the trajectory noise with good observations.

In the case of the EKF, the squared error has changed considerably, increasing very rapidly to ϵ_{EKF2} $= 13967.03 >>> \epsilon_{EKF1}$. The trajectory and observation curves coincide, but the estimated trajectory is very different from the others. The EKF is much more sensitive to trajectory noise than the KF.

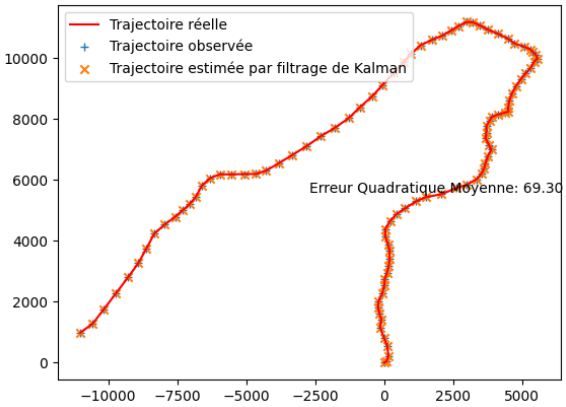

Figure 3: KF trajectories noisy trajectory scenario

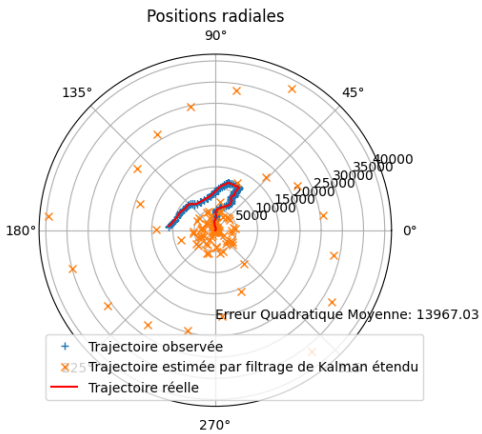

Figure 4: EKF trajectories noisy trajectory scenario

3. Noisy Observation in x-axis Scenario

Let us consider the case $(\sigma_Q, \sigma_{px}, \sigma_{py}) = (1,\ 300,\ 3)$ by varying the observation noise along the x-axis.

The trajectory is linear because there is no more trajectory noise. In the KF case, there is an error between the real and estimated trajectories of $\epsilon_{KF3} = 40.56$, but they are close to each other unlike the observed trajectory, which is very different from them. To determine the cause of ϵ_{KF3}, we decide to observe the y-coordinates and x-coordinates of the trajectories separately. The y-coordinate curves are much more correlated with each other with a low error of $\epsilon_y = 26.49$ than the x-coordinate curves with a high error of $\epsilon_x = 561.37$. The error therefore comes from the x-coordinates: the observation error in x implies an estimation error of the trajectory! We note that although $\epsilon_x = 561.37$, a very good observation of the y-coordinates allows the errors of the estimated trajectory to be reduced to only $\epsilon_{KF3} = 40.56$, making the KF more sensitive to trajectory noise than to observation noise along the x-axis. We can roughly attempt to correct the estimated trajectory by shifting it left or right.

In the case of the EKF, the squared error $\epsilon_{EKF3} = 3580.89$ has decreased significantly compared to the noisy trajectory case, but remains very high compared to the ideal case. The trajectory and observation curves coincide, but not the estimated trajectory, which remains just as unusable. Although highly sensitive to observation noise, the EKF equations are much more sensitive to trajectory noise.

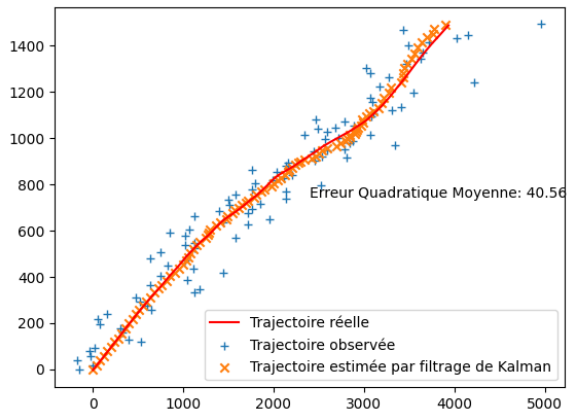

Figure 5: KF trajectories noisy x-axis observation scenario

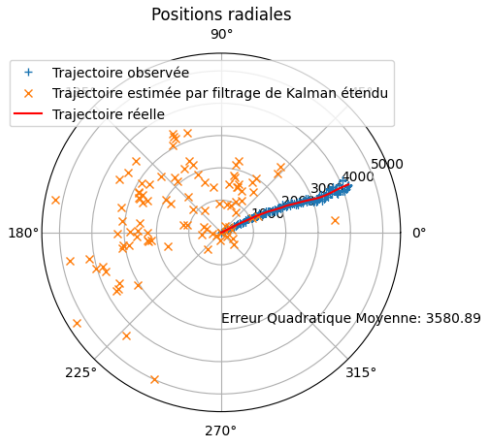

Figure 6: EKF trajectories noisy x-axis observation scenario

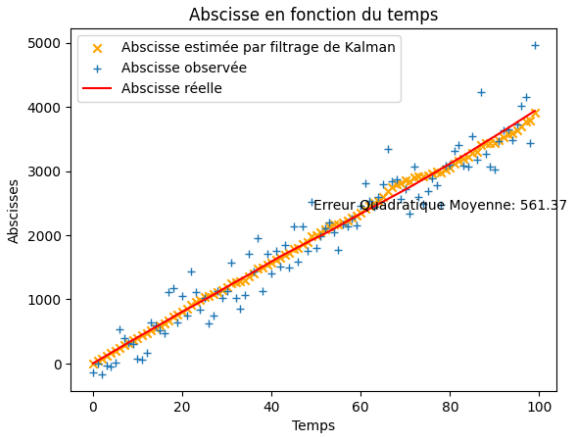

Figure 7: KF x-coordinates noisy x-axis observation scenario

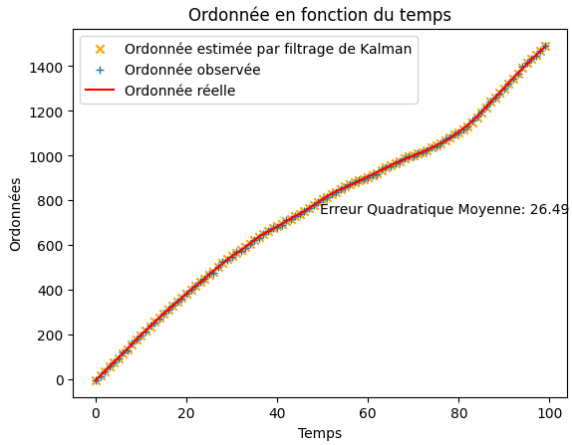

Figure 8: KF y-coordinates noisy x-axis observation scenario

4. Noisy Observation in y-axis Scenario

Let us consider the case $(\sigma_Q, \sigma_{px}, \sigma_{py}) = (1,\ 3,\ 300)$ by varying the observation noise along the y-axis. Applying analogous reasoning to the y-coordinates, we arrive at the same conclusions.

5. Scenario Varying the Sampling Frequency

Setting $(\sigma_Q, \sigma_{px}, \sigma_{py}) = (1,\ 30,\ 30)$, we measured the mean squared errors as a function of different sampling periods. For the KF, we observe that the longer the sampling period, the fewer points there are, and the greater the error, hence the lower the filtering quality. For the EKF, the trend is reversed. We can interpret this result by assuming that the fewer points we take, the less we rely on the approximation, hence the less error there is. However, regardless of the sampling period, the KF is significantly better than the EKF.

	$T_e = 0.5$	$T_e = 1$	$T_e = 2$	$T_e = 5$	$T_e = 10$
KF	13.54	16.59	23.10	30.40	28.98
EKF	3790.33	1456.61	1160.50	722.20	556.70

Table 1: Mean squared errors of the KF and EKF for different sampling periods

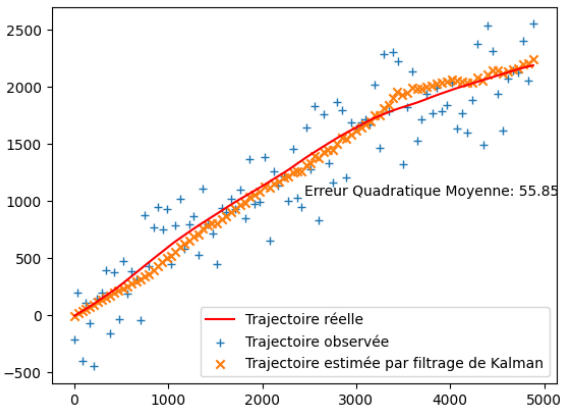

Figure 9: KF trajectories noisy y-axis observation scenario

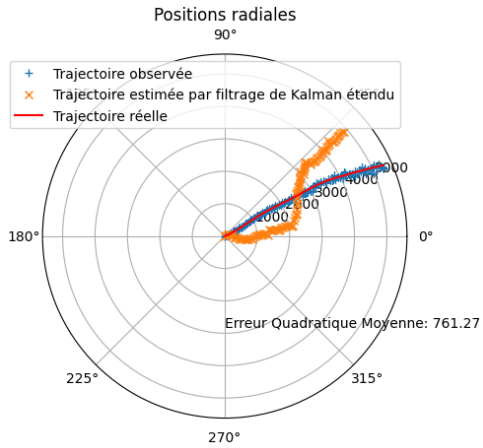

Figure 10: EKF trajectories noisy y-axis observation scenario

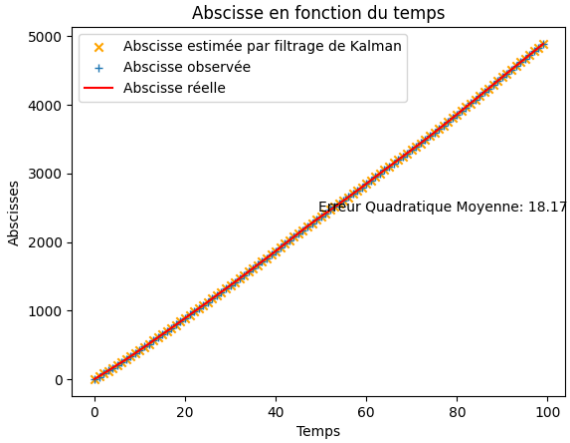

Figure 11: KF x-coordinates noisy y-axis observation scenario

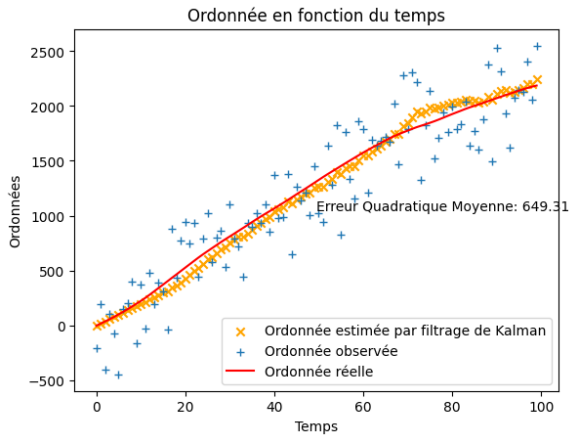

Figure 12: KF y-coordinates noisy y-axis observation scenario

6. Conclusion

On the one hand, the KF presents low errors compared to the EKF since the EKF is only a local approximation of the problem, which becomes poor globally. It is not very sensitive to observation noise but is somewhat more sensitive to trajectory noise. On the other hand, the EKF is very sensitive to all types of noise, in particular to trajectory noise, to which the remainder of the Taylor expansion will always be added in the form of observation noise.

$(\sigma_Q, \sigma_{px}, \sigma_{py})$	(1, 3, 3)	(100, 3, 3)	(1, 300, 3)	(1, 3, 300)
KF	3.36	69.30	40.56	55.85
EKF	491.33	13967.03	3580.89	761.27

Table 2: Summary table of mean squared errors for the first four scenarios

4. Appendix: Python Code

3.1.2

```
import numpy as np
import matplotlib.pyplot as plt
import math
```

```python

T_e = 1
T = 100
sigma_Q = 1
sigma_px = 30
sigma_py = 30

F = np.array([[1,T_e,0,0],
              [0,1,0,0],
              [0,0,1,T_e],
              [0,0,0,1]])
H = np.array([[1,0,0,0],
              [0,0,1,0]])

Q = np.array([[(T_e**3)/3, (T_e**2)/2, 0, 0 ],
              [(T_e**2)/2, T_e, 0,0],
              [0,0,(T_e**3)/3, (T_e**2)/2],
              [0,0,(T_e**2)/2, T_e]])
Q = (sigma_Q**2)*Q

R = np.array([[sigma_px**2, 0],
              [0, sigma_py**2]])

x_init = np.transpose(np.array([3, 40, -4, 20]))

x_kalm = x_init

P_kalm = np.eye(4)

meanQ = np.zeros((4,1))

meanR = np.zeros((2,1))
```

Source 1: Python code to initialise the problem parameters

3.1.3

```python
def creer_trajectoire(F, Q, x_init, T):
    vecteur_x = np.zeros((4, math.floor(T/T_e)))
    vecteur_x[:, 0] = x_init

    for k in range(1, math.floor(T/T_e)):
        u = np.random.multivariate_normal(mean=np.zeros(4), cov=Q)
        vecteur_x[:, k] = np.dot(F, vecteur_x[:, k-1]) + u

    return vecteur_x
```

Source 2: Python code to generate a trajectory from the initial position

3.1.4

```python
def creer_observations(H, R, vecteur_x, T):
    vecteur_y = np.zeros((2, math.floor(T/T_e)))
```

```python

    for k in range(math.floor(T/T_e)):
        v = np.random.multivariate_normal(mean=np.zeros(2), cov=R)
        vecteur_y[:, k] = np.dot(H, vecteur_x[:, k]) + v

    return vecteur_y
```

Source 3: Python code to generate an observation of a trajectory

3.1.5

```python
X1 = creer_trajectoire(F,Q,x_init,T) #real trajectory
X2 = creer_observations(H,R,X1,T) #observed trajectory

plt.figure()
plt.plot(X1[0],X1[2],'r', label = "Real trajectory")
plt.scatter(X2[0],X2[1],marker = "+", linewidth =1, label = "Observed trajectory")
plt.legend()
plt.show()
```

Source 4: Python code to plot the real trajectory of a target and its observed trajectory

3.1.6

```python
def kalman_filter(F, Q, H, R, y_k, x_kalm_prec, P_kalm_prec):
    # Prediction
    x_kalm_pred = np.dot(F, x_kalm_prec)
    P_kalm_pred = np.dot(np.dot(F, P_kalm_prec), F.T) + Q

    # Update
    K = np.dot(np.dot(P_kalm_pred, H.T), np.linalg.inv(np.dot(np.dot(H, P_kalm_pred), H.T) + R))
    x_kalm_k = x_kalm_pred + np.dot(K, y_k - np.dot(H, x_kalm_pred))
    P_kalm_k = np.dot((np.eye(len(F)) - np.dot(K, H)), P_kalm_pred)

    return x_kalm_k, P_kalm_k
```

Source 5: Python code implementing the classical Kalman filter

3.1.7

```python
#To be written after writing 3.1.8
y_k = X2
x_kalm_prec = x_init
P_kalm_prec = np.eye(4)

res = np.zeros((4,math.floor(T/T_e)))
res[:,0] = x_kalm_prec


for k in range(1, math.floor(T/T_e)):
```

```
    x_kalm_k, P_kalm_k = kalman_filter(F, Q, H, R, y_k[:, k],
        x_kalm_prec, P_kalm_prec)
    res[:, k] = x_kalm_k
    x_kalm_prec = x_kalm_k
    P_kalm_prec = P_kalm_k

erreur = erreur_quadratique_moyenne(X1, res)

plt.figure()
plt.plot(X1[0],X1[2],'r', label = "Real trajectory")
plt.scatter(X2[0],X2[1],marker = "+", linewidth =1, label = "Observed
    trajectory")
plt.scatter(res[0,:], res[2,:],marker = 'x', label = "Trajectory
    estimated by Kalman filtering")
plt.legend()
plt.text(0.5, 0.5, f'Mean Squared Error: {erreur:.2f}',
    transform=plt.gca().transAxes, fontsize=10)
plt.show()
```

Source 6: Python code to plot the estimated, observed, and real trajectories

3.1.8

```
def err_quadra(k):#squared error of the k-th measurement
  Xk = X1[:,k] #trajectory
  Xkk = res[:,k]
  X = Xk-Xkk
  return np.dot(np.transpose(X),X)

def erreur_moyenne(k):#mean squared error of the first k measurements
  L=[err_quadra(j) for j in range(k)]
  return (1/math.floor(T/T_e))*(sum(np.sqrt(L)))
```

Source 7: Python code to measure the squared error between the real and estimated trajectories

3.1.8.bis

```
def erreur_quadratique(vecteur1, vecteur2):#We take two vectors as
    input
    erreur = vecteur1 - vecteur2
    return np.dot(np.transpose(erreur), erreur)

def erreur_quadratique_moyenne1(vecteur1, vecteur2):
    return np.mean(np.sqrt(erreur_quadratique(vecteur1, vecteur2)))

def erreur_quadratique_moyenne(vecteurs1, vecteurs2):
    nb_vecteurs = len(vecteurs1[0])
    erreurs = [erreur_quadratique(vecteurs1[:,i], vecteurs2[:,i]) for
        i in range(1, nb_vecteurs)]
    erreur_moyenne = np.mean(np.sqrt(erreurs))
    return erreur_moyenne
```

Source 8: Python code to measure the squared error between the real and estimated trajectories directly from two arrays of vectors

3.1.9

```
Temps = np.arange(0,  math.floor(T/T_e))
Temps = Temps*T_e
print(Temps)

erreurx = erreur_quadratique1(res[0, :],X1[0, :])
erreury = erreur_quadratique1(res[2, :],X1[2, :])

# Plot of the x-coordinate as a function of time
plt.figure()
plt.scatter(Temps, res[0, :], marker='x', color='orange',
    label="x-coordinate estimated by Kalman filtering")
plt.scatter(Temps,X2[0, :], marker="+", linewidth=1, label="Observed
    x-coordinate")
plt.plot(Temps, X1[0, :], 'r', label="Real x-coordinate")
plt.xlabel("Time")
plt.ylabel("x-coordinates")
plt.title("x-coordinate as a function of time")
plt.legend()
plt.text(0.5, 0.5, f'Mean Squared Error: {erreurx:.2f}',
    transform=plt.gca().transAxes, fontsize=10)
plt.show()

# Plot of the y-coordinate as a function of time
plt.figure()
plt.scatter(Temps,res[2, :], marker='x', color='orange',
    label="y-coordinate estimated by Kalman filtering")
plt.scatter(Temps,X2[1, :], marker="+", linewidth=1, label="Observed
    y-coordinate")
plt.plot(Temps,X1[2, :], 'r', label="Real y-coordinate")
plt.xlabel("Time")
plt.ylabel("y-coordinates")
plt.title("y-coordinate as a function of time")
plt.legend()
plt.text(0.5, 0.5, f'Mean Squared Error: {erreury:.2f}',
    transform=plt.gca().transAxes, fontsize=10)
plt.show()
```

Source 9: Python code to plot the x and y coordinates of the target as a function of time

3.2.1

```
def kalman_filter(F, Q, H, R, y_k, x_kalm_prec, P_kalm_prec):
    # Prediction
    x_kalm_pred = np.dot(F, x_kalm_prec)
    P_kalm_pred = np.dot(np.dot(F, P_kalm_prec), F.T) + Q

    # Check if the observation is valid (not NaN)
    if not np.isnan(y_k).any():
        # Update
        K = np.dot(np.dot(P_kalm_pred, H.T),
            np.linalg.inv(np.dot(np.dot(H, P_kalm_pred), H.T) + R))
```

```python
        x_kalm_k = x_kalm_pred + np.dot(K, y_k - np.dot(H, x_kalm_pred))
        P_kalm_k = np.dot((np.eye(len(F)) - np.dot(K, H)), P_kalm_pred)
    else:
        # In case of non-detection, use the prediction
        x_kalm_k = x_kalm_pred
        P_kalm_k = P_kalm_pred

    return x_kalm_k, P_kalm_k

def kalman_filter1(F, Q, H, R, y_k, x_kalm_prec, P_kalm_prec):
    # Prediction
    x_kalm_pred = np.dot(F, x_kalm_prec)
    P_kalm_pred = np.dot(F @ P_kalm_prec, F.T) + Q

    # Check if the observation is valid (not NaN)
    if np.all(~np.isnan(y_k)):
        # Normal update
        S = H @ P_kalm_pred @ H.T + R
        K = P_kalm_pred @ H.T @ np.linalg.inv(S)
        x_kalm_k = x_kalm_pred + K @ (y_k - H @ x_kalm_pred)
        P_kalm_k = (np.eye(len(x_kalm_prec)) - K @ H) @ P_kalm_pred
    else:
        # Skip the update in case of non-detection
        x_kalm_k = x_kalm_pred
        P_kalm_k = P_kalm_pred

    return x_kalm_k, P_kalm_k
```

Source 10: Python code implementing the Kalman filter in the absence of observation

4.3

```python
sigma_angle = np.pi/180
sigma_dist = 10
R = np.array([[sigma_angle**2, 0],
              [0, sigma_dist**2]])

def creer_observations_radar(R, vecteur_x):
    vecteur_y = np.zeros((2, vecteur_x.shape[1]))

    for k in range(vecteur_x.shape[1]):
        px, py = vecteur_x[0, k], vecteur_x[2, k]

        # Conversion to polar coordinates
        angle = np.arctan2(py, px)
        distance = np.sqrt(px**2 + py**2)

        # Adding radar noise and creating the observation
        v = np.random.multivariate_normal(mean=np.zeros(2), cov=R)
        vecteur_y[:, k] = np.array([[angle], [distance]]).flatten() + v #dimension issues resolved with .flatten()

```

```
    return vecteur_y
```

Source 11: Python code to convert a Cartesian observation to a polar observation

4.6

```
def H_prim(x_kalm_pred):
    px, py = x_kalm_pred[0], x_kalm_pred[2]
    res = np.array([[px/np.sqrt(px**2 + py**2), 0, py/np.sqrt(px**2 + py**2), 0],
                    [-py/(px**2 + py**2), 0, px/(px**2 + py**2), 0]])
    return res

def h(x_kalm_pred):
    px, py = x_kalm_pred[0], x_kalm_pred[2]
    res = np.array([[np.sqrt(px**2 + py**2)],
                    [np.arctan2(py, px)]])
    return res

def filtre_de_kalman_radar(F, Q, R, y_k, x_kalm_prec, P_kalm_prec):
    # Prediction
    x_kalm_pred = np.dot(F, x_kalm_prec)
    P_kalm_pred = np.dot(np.dot(F, P_kalm_prec), F.T) + Q

    H = H_prim(x_kalm_pred)
    z_k = y_k - np.squeeze(h(x_kalm_pred)) + np.dot(H, x_kalm_pred)

    # Update
    K = np.dot(np.dot(P_kalm_pred, H.T), np.linalg.inv(np.dot(np.dot(H, P_kalm_pred), H.T) + R))
    x_kalm_k = x_kalm_pred + np.dot(K, z_k - np.dot(H, x_kalm_pred))
    P_kalm_k = np.dot((np.eye(len(F)) - np.dot(K, H)), P_kalm_pred)

    return x_kalm_k, P_kalm_k
```

Source 12: Python code implementing the extended Kalman filter

4.7

```
y_k = X2 #observations in Cartesian coordinates
x_kalm_prec = x_init
P_kalm_prec = np.eye(4)

res = np.zeros((4,math.floor(T/T_e)))
res[:,0] = x_kalm_prec

for k in range(1, math.floor(T/T_e)):
    x_kalm_k, P_kalm_k = filtre_de_kalman_radar(F, Q, R, y_k[:,k], x_kalm_prec, P_kalm_prec)
    res[:, k] = x_kalm_k
    x_kalm_prec = x_kalm_k
    P_kalm_prec = P_kalm_k

```

```python
angles_obs, rayons_obs = creer_observations_radar(R, X1)[0], creer_observations_radar(R, X1)[1] #X1 is the real trajectory
rayons_est, angles_est = np.sqrt(res[0]**2+res[2]**2) , np.arctan2(res[2], res[0])
rayons_rel, angles_rel = np.sqrt(X1[0]**2+X1[2]**2) , np.arctan2(X1[2], X1[0])

erreur = erreur_quadratique_moyenne(X1,res)

plt.figure()
plt.polar(angles_obs, rayons_obs, marker='+', label = "Observed trajectory",linestyle='None')
plt.polar(angles_est, rayons_est, marker="x", label = "Trajectory estimated by extended Kalman filtering" ,linestyle = "None")
plt.polar(angles_rel, rayons_rel, color = 'red', label = "Real trajectory")
plt.title('Polar positions')
plt.legend()
plt.text(0.5, 0.25, f'Mean Squared Error: {erreur:.2f}', transform=plt.gca().transAxes, fontsize=10)
plt.show()
```

Source 13: Python code to plot the observed, estimated, and real positions